✈ Airport Booking Portal

By Isha Saleem

📌 Project Overview

The **Airport Booking Portal** is a role-based frontend web application built using **React.js**. It simulates a real-world airline booking system where users can register, log in, search flights, book tickets, and manage bookings based on their assigned roles.

The system supports three user roles:

- **Passenger**
- **Employee**
- **Admin**

All data is managed using **localStorage**, simulating backend behavior without server-side integration.

🎯 Objectives

- Build a role-based frontend application using React
- Simulate real-world airport booking workflow
- Implement authentication and authorization (frontend-based)
- Practice component-based architecture
- Handle CRUD operations using localStorage
- Design a modern responsive UI

👥 User Roles & Permissions

🧳 Passenger

- View available flights
- Book flights

- View personal bookings
- Cancel bookings
- Edit profile
- Contact support

👨💼 Employee

- View all bookings
- Approve / Reject / Complete bookings
- Manage booking status
- View dashboard stats

🛠️ Admin

- Add new flights
- Delete flights
- Manage users
- View all bookings
- Access system dashboard
- Full control over system data

⚙️ Core Features

🔐 Authentication System

- Login / Register system
- Role-based login redirection
- Session stored in localStorage

✈️ **Flight Management**

- Add flights (Admin only)
- Delete flights (Admin only)
- View available flights (Passenger/Employee/Admin)
- Search flights by origin/destination

🎫 **Booking System**

- Book flights (Passenger)
- Store bookings in localStorage
- Cancel booking (Passenger)
- Update booking status (Employee/Admin)

📊 Dashboard System

Role-based analytics:

Passenger Dashboard

- My bookings count
- Upcoming flights

Employee Dashboard

- Total bookings
- Pending requests

Admin Dashboard

- Total users
- Total bookings
- Total flights

👥 User Management (Admin)

- View all registered users
- Delete users
- Role-based control

👤 Profile Management

- View user profile
- Edit name/email
- Save updates to localStorage

📞 Contact Support Page

- Role-based support information
- Static support contact details
- Category-based help sections

🧭 Navigation System

- Dynamic sidebar based on role
- Active route highlighting
- React Router-based navigation

🏗 System Architecture

📂 Folder Structure

```
project/
|
 ├── src/
```

```
|   ├── assets/
|   ├── components/
|   ├── context/
|   ├── layouts/
|   |   └── MainLayout.jsx
|   ├── pages/
|   |   ├── auth/
|   |   |   ├── Login.jsx
|   |   |   └── Register.jsx
|   |   ├── passenger/
|   |   |   ├── Flights.jsx
|   |   |   └── Booking.jsx
|   |   ├── employee/
|   |   |   └── ManageBookings.jsx
|   |   ├── admin/
|   |   |   ├── Users.jsx
|   |   |   ├── ManageFlights.jsx
|   |   |   └── AdminDashboard.jsx
|   |   └── common/
|   |       ├── Profile.jsx
|   |       └── Contact.jsx
|   |
|   ├── routes/
|   |   └── ProtectedRoute.jsx
|   ├── App.jsx
|   └── main.jsx
|
└── public/
```

🧠 State Management

- **React useState & useEffect**
- **localStorage used as pseudo-backend**

Stored Data Keys:

- user → logged-in user session

- allUsers → registered users list
- adminFlights → flights created by admin
- bookings → flight bookings

🔐 Authentication Flow

1. User registers → saved in allUsers
2. User logs in → validated from localStorage
3. User session stored in user
4. Role determines dashboard access
5. Protected routes restrict unauthorized access

🛡 Route Protection

- ProtectedRoute ensures login required
- Role-based access control (RBAC)
- Admin / Employee route guards

🎨 UI/UX Design

Design Features:

- Modern airport-themed UI
- Glassmorphism login/register pages
- Sidebar navigation
- Responsive layout support
- Card-based dashboards
- Gradient and shadow-based design

📱 Responsiveness

- Sidebar hidden on mobile
- Grid layouts adapt to screen size
- Flexible header layout
- Mobile-friendly input fields

⚙️ Technologies Used

- React.js (Functional Components)
- React Router DOM
- JavaScript (ES6+)
- HTML5 & CSS3
- Inline Styling
- localStorage (Mock Backend)

🚀 How to Run Project

clone repository

npm install
npm start

🏁 Conclusion

This project demonstrates a complete **frontend-based airport booking simulation system** with role-based access control and real-world workflow simulation.

It showcases:

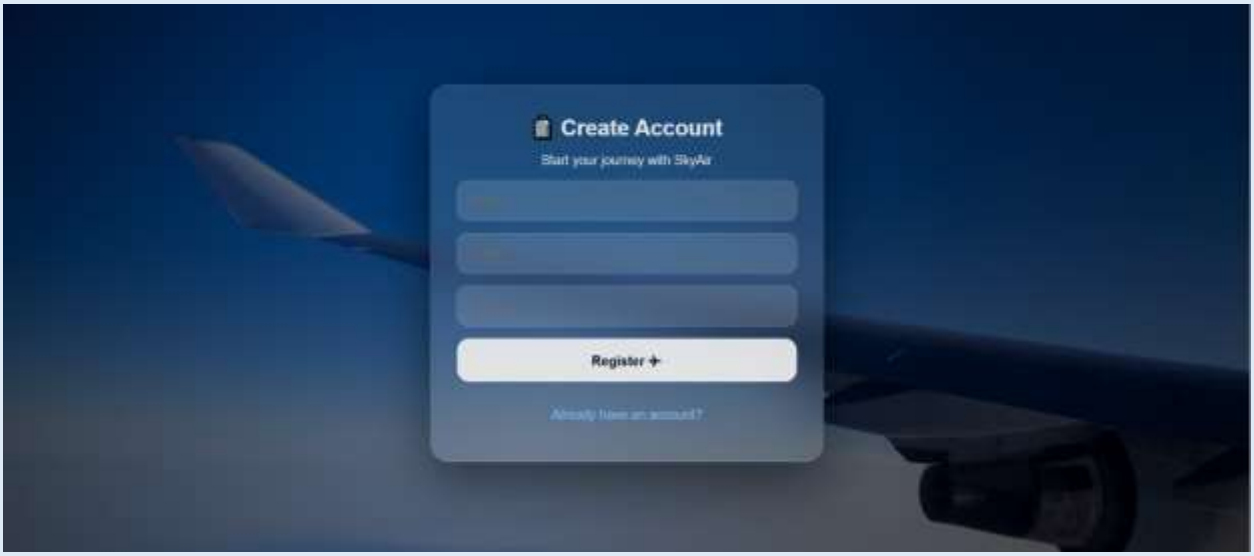
- React component architecture
- State management using hooks

- CRUD operations using localStorage
- Role-based UI rendering
- Practical UI/UX design implementation

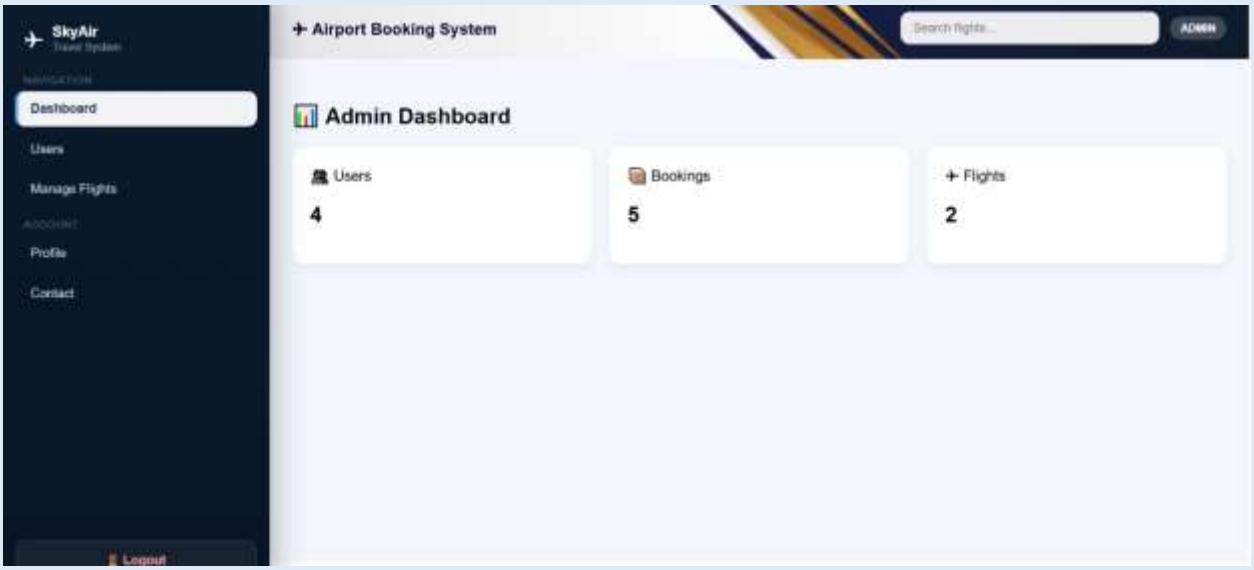
Mockups:

Login :

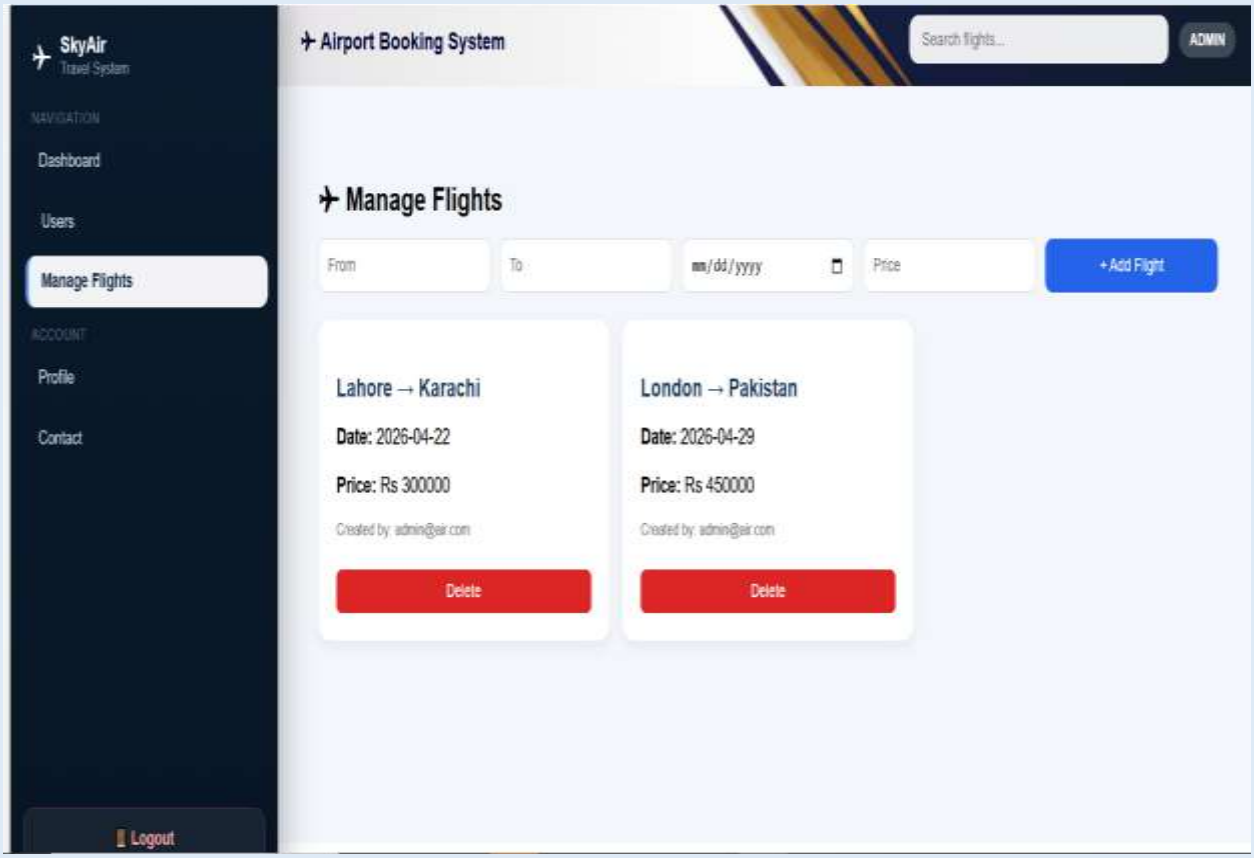
Signup :



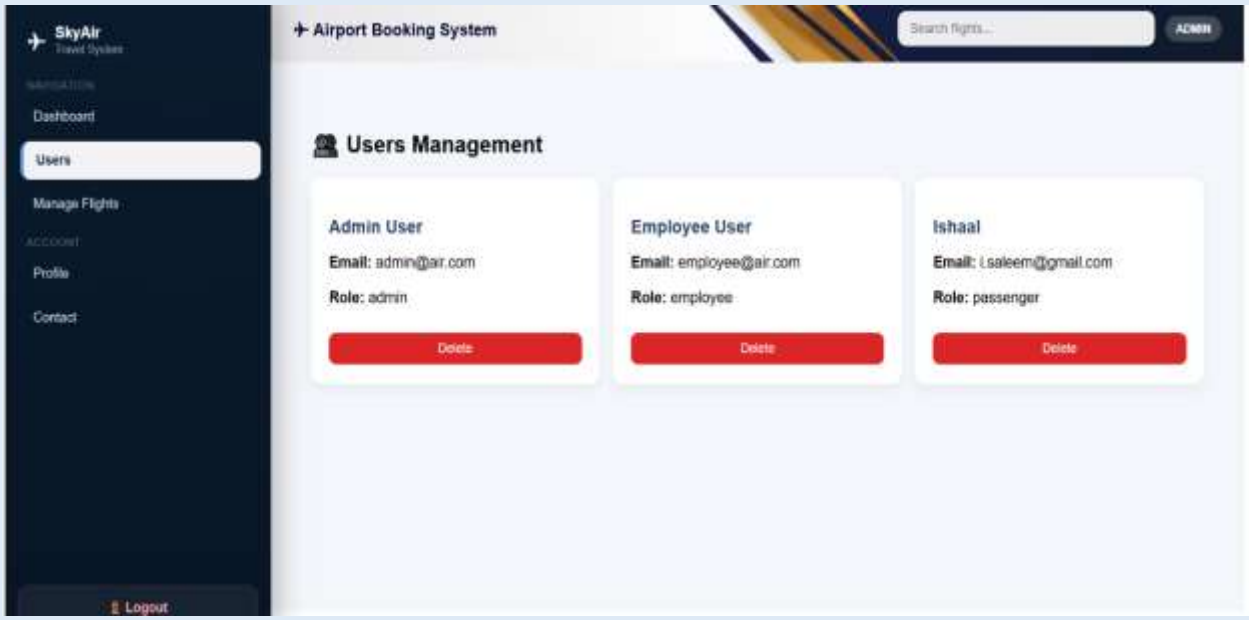
Admin Dashboard:



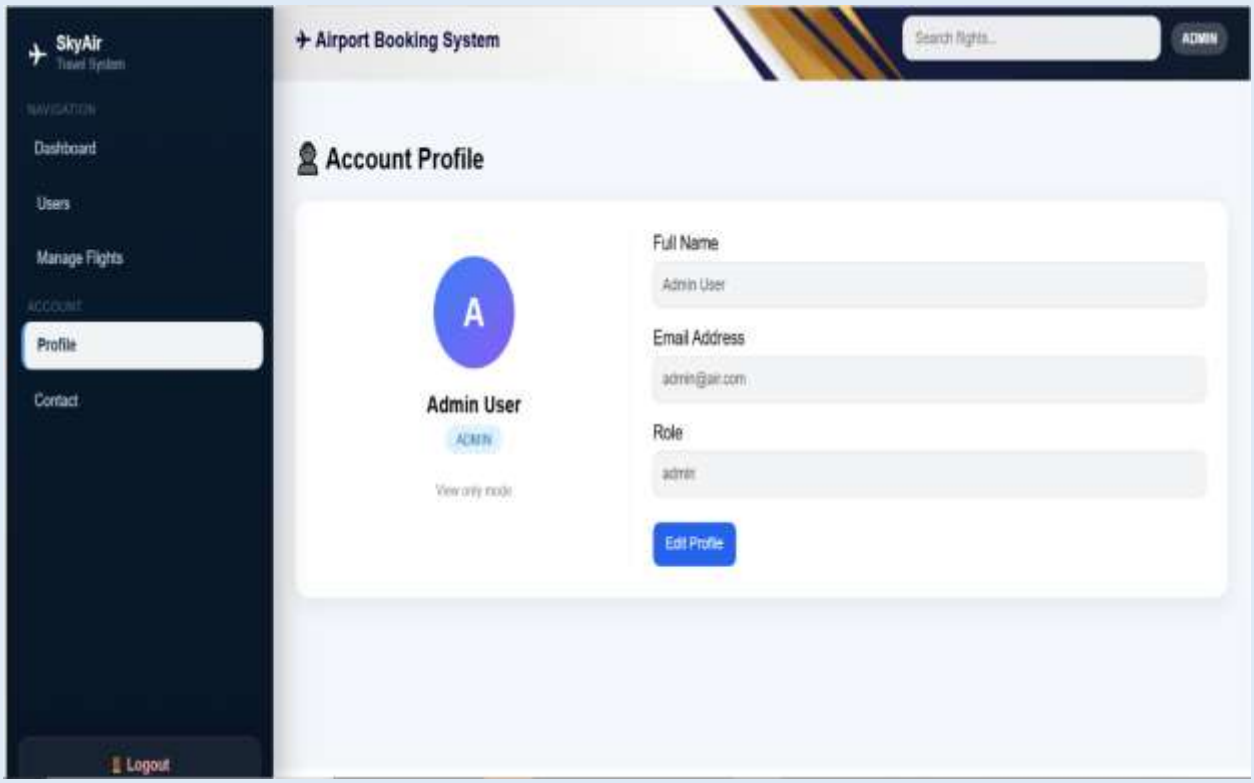
Admin flights management:



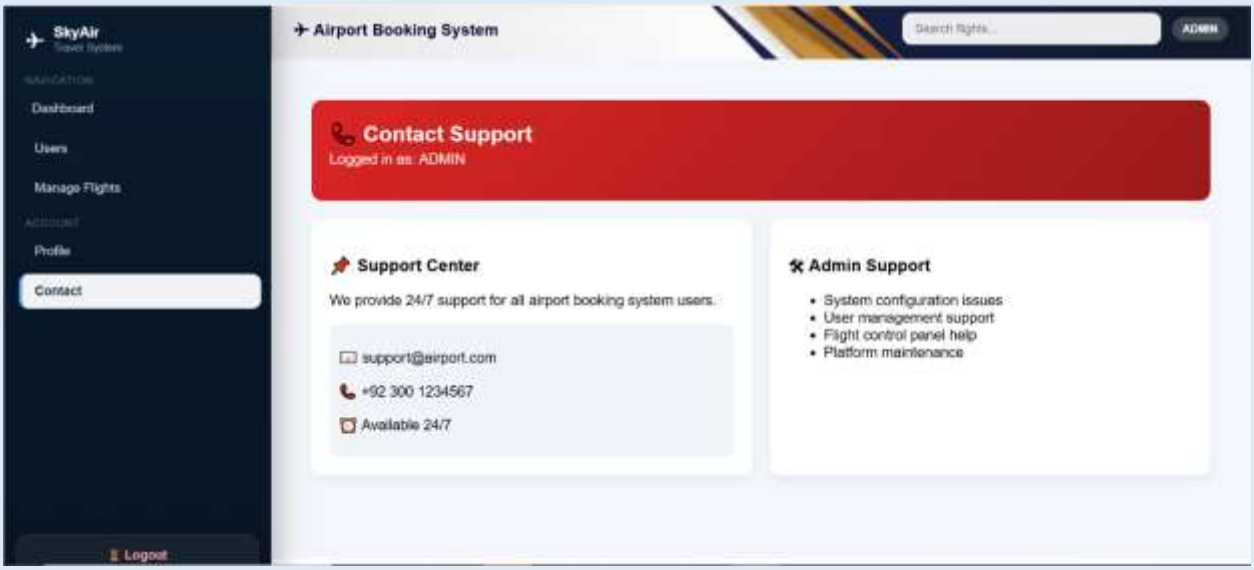
Admin User Management:



Admin profile view:

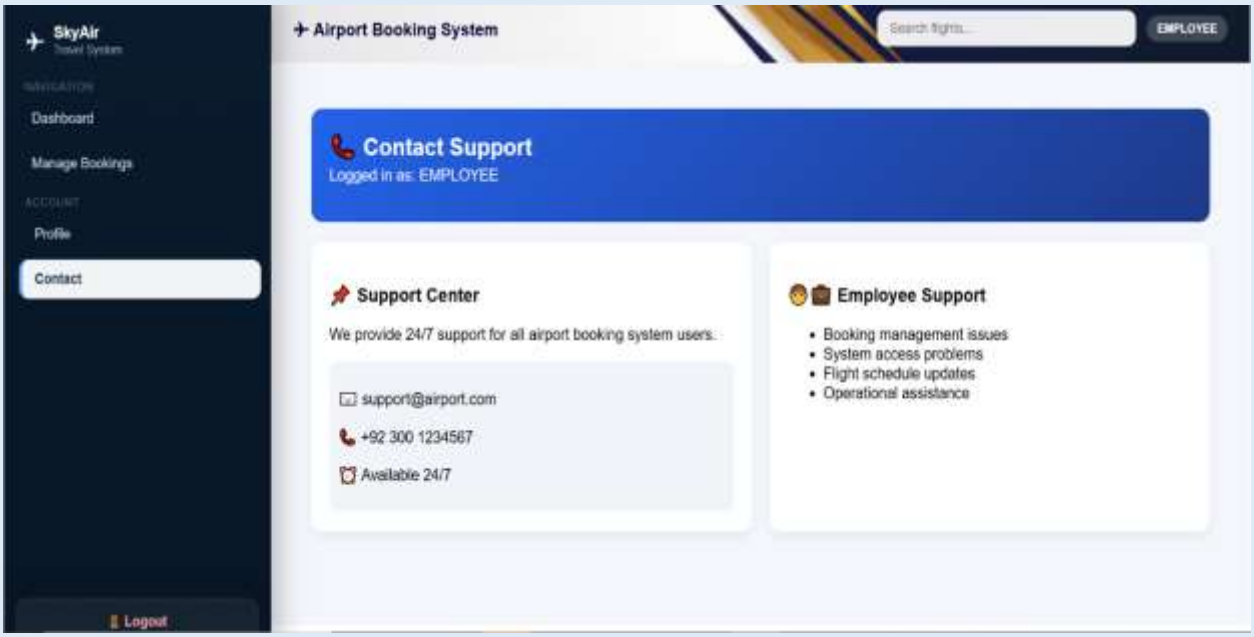


Contact view(Admin):

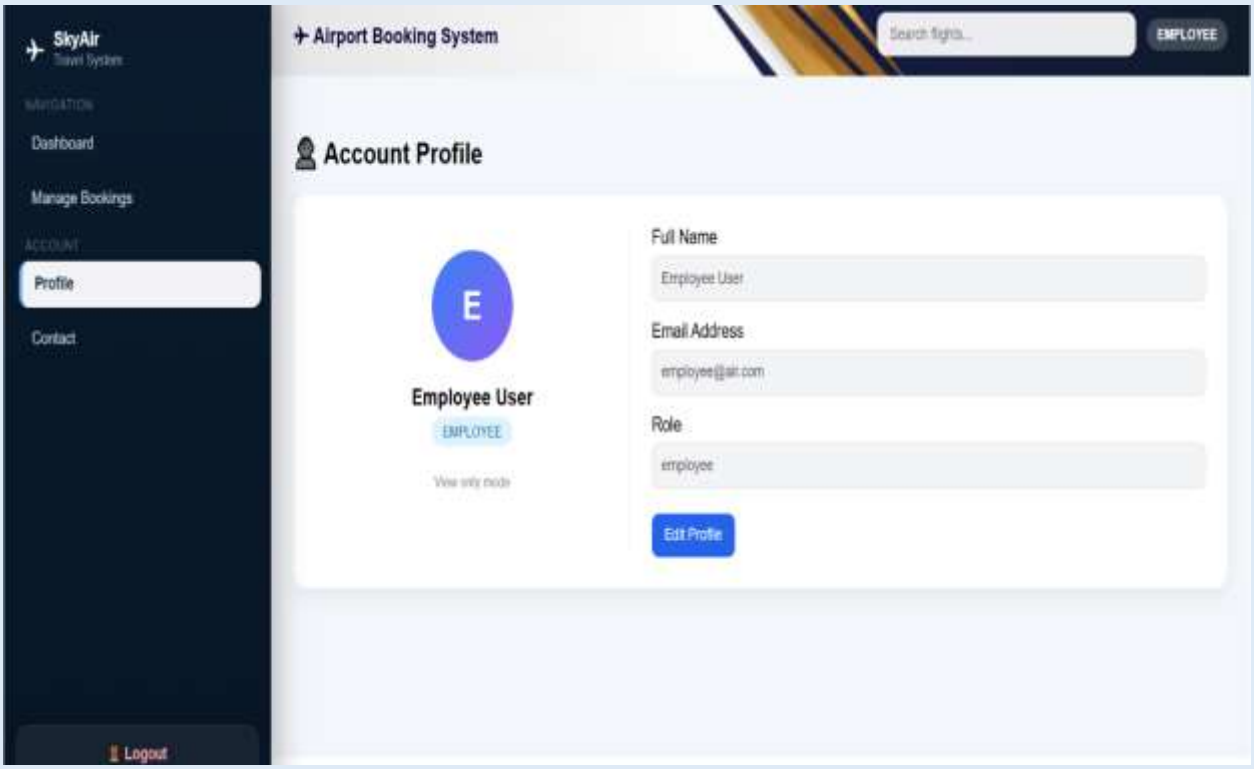


Employee Booking Management:

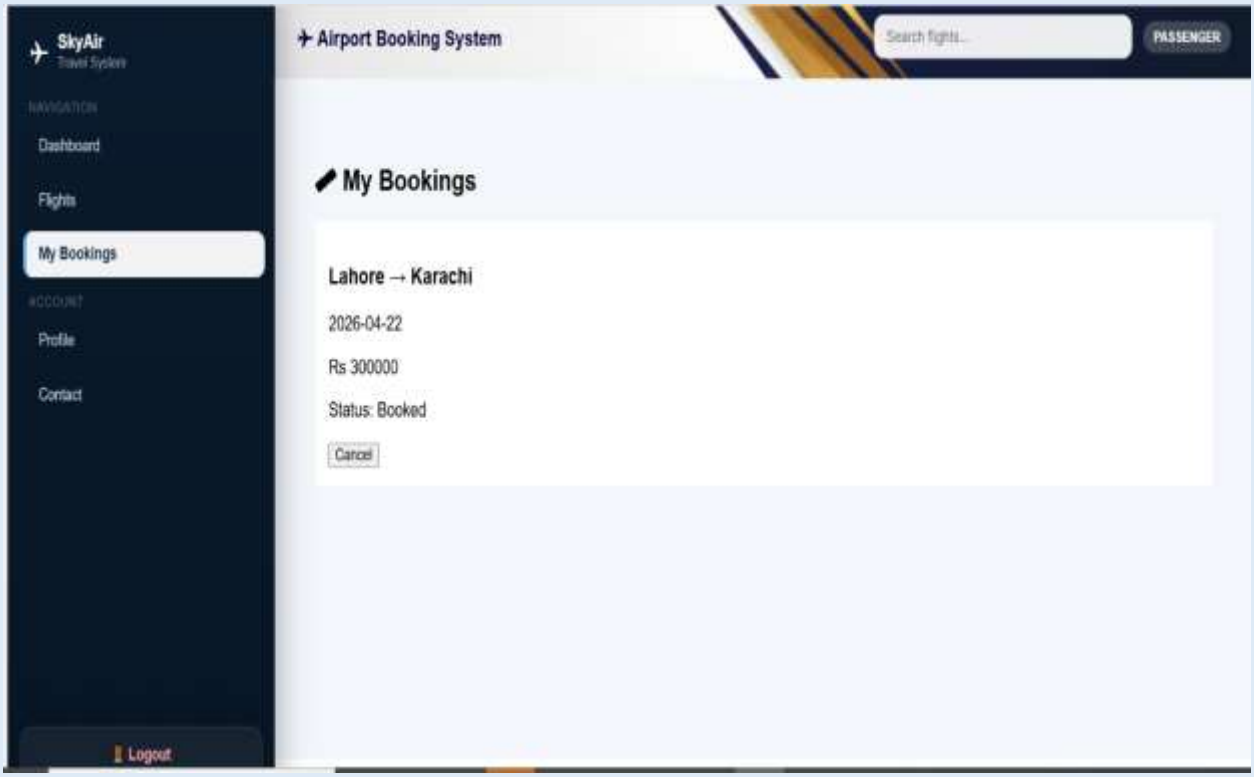
Contact view(Employee):



Employee Profile view:

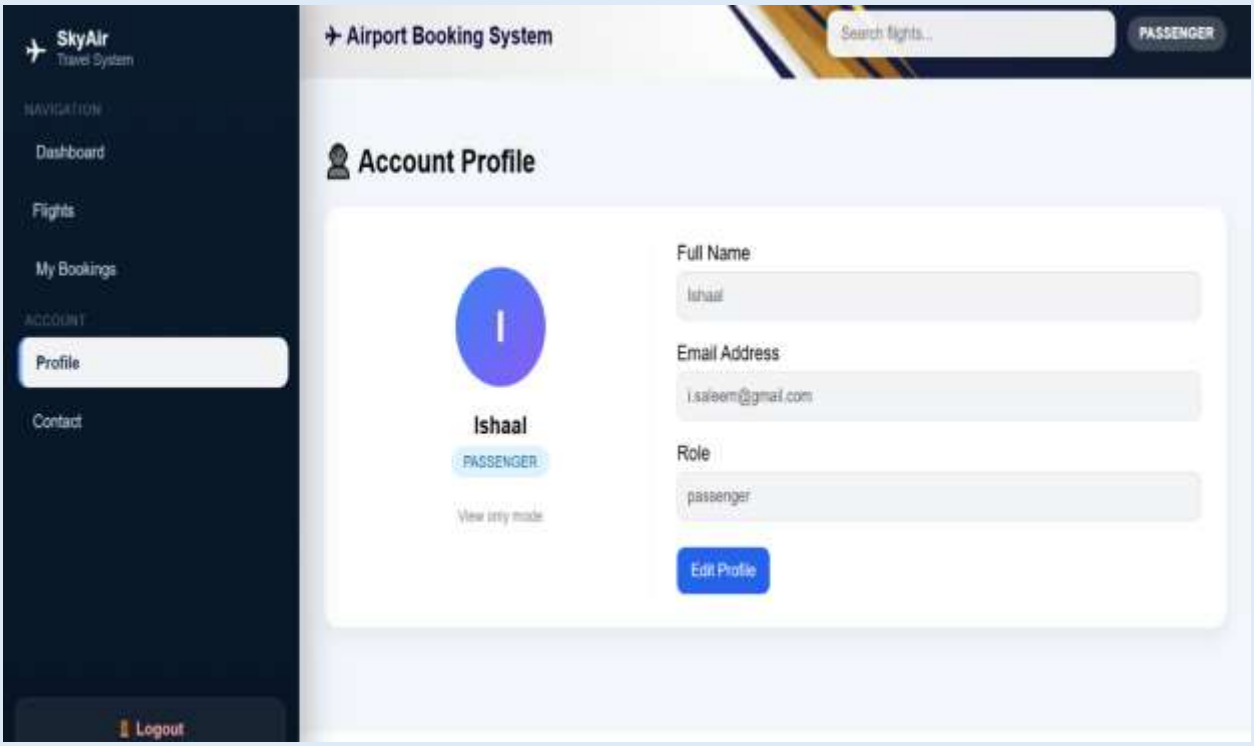
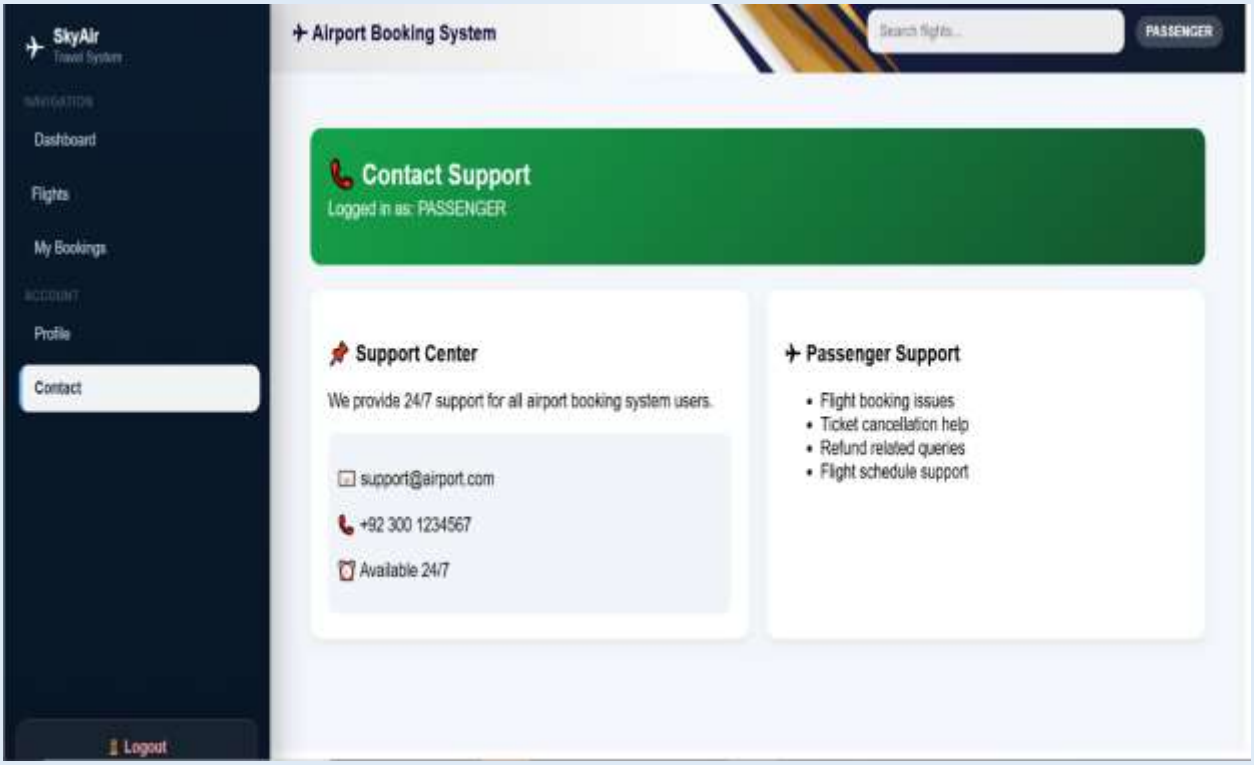


User Bookings:



User Flight view:

User Profile view:



Contact View(Users):